

HerAI Fellowship

DATA ENGINEERING

Modul Belajar Praktis dan Mudah Dipahami

Membangun aliran data yang andal dari sumber hingga siap dipakai analitik dan AI.

Level	Intermediate
Estimasi belajar	18-22 jam
Prasyarat	Python dasar, SQL dasar, dan pemahaman data tabular
Versi	1.0 - 23 Juli 2026
Route	/participant/courses/data-engineering

Disusun untuk Participant Module - Project HerAI

Tentang Modul

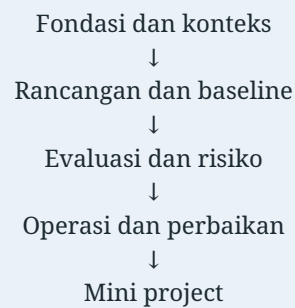
Data Engineering membahas cara merancang, membangun, menguji, mengamankan, dan mengoperasikan pipeline data. Peserta belajar melihat data sebagai produk yang memiliki kontrak, pemilik, kualitas, lineage, dan target layanan.

Pola belajar: pahami tujuan, buat baseline kecil, uji kegagalan, dokumentasikan bukti, lalu perbaiki secara bertahap. Jangan menambah kompleksitas sebelum masalah dan metriknya jelas.

Cara Menggunakan Modul

- Baca tujuan dan gambaran sederhana setiap bab.
- Kerjakan checkpoint tanpa melihat kembali materi.
- Adaptasi contoh pada data aman atau sintetis.
- Catat asumsi, error, keputusan, dan keterbatasan.
- Gunakan mini project sebagai artefak portofolio.

Peta Belajar



Daftar Isi

- 01 Bab 1 - Peran dan Arsitektur Data Engineering**
- 02 Bab 2 - Sumber Data, Format, dan Data Contract
- 03 Bab 3 - Batch Processing dan Stream Processing
- 04 Bab 4 - ETL, ELT, dan Transformasi Data
- 05 Bab 5 - Data Warehouse, Data Lake, dan Lakehouse
- 06 Bab 6 - Orkestrasi dan Penjadwalan Pipeline
- 07 Bab 7 - Kualitas Data dan Pengujian
- 08 Bab 8 - Metadata, Lineage, dan Governance
- 09 Bab 9 - Keamanan dan Privasi Data
- 10 Bab 10 - Observability, Reliability, dan Biaya
- 11 Bab 11 - Mini Project: Pipeline Analitik Penjualan**
- 12 Bab 12 - Kuis Akhir**
- 13 Bab 13 - Diskusi dan Refleksi
- 14 Bab 14 - Glosarium dan Checklist
- 15 Bab 15 - Ringkasan dan Referensi**

HerAI Fellowship - Participant Module Membangun aliran data yang andal dari sumber hingga siap dipakai analitik dan AI.

Tentang Modul

Data Engineering membahas cara merancang, membangun, menguji, mengamankan, dan mengoperasikan pipeline data. Peserta belajar melihat data sebagai produk yang memiliki kontrak, pemilik, kualitas, lineage, dan target layanan.

Modul disusun dengan bahasa bertahap. Setiap bab berisi tujuan, konsep inti, alur kerja, contoh kasus, kesalahan umum, checkpoint, dan latihan. Peserta tidak perlu menghafal seluruh istilah. Yang lebih penting adalah memahami hubungan antarkomponen dan mampu menjelaskan alasan di balik sebuah pilihan.

Capaian Pembelajaran

Setelah menyelesaikan modul, peserta mampu:

- menjelaskan fondasi dan istilah utama Data Engineering;
- menerjemahkan kebutuhan menjadi rancangan yang dapat diuji;
- membuat prototipe kecil dengan dokumentasi dan kontrol dasar;
- mengevaluasi kualitas, risiko, keterbatasan, serta biaya;
- menyampaikan hasil dan rekomendasi kepada pemangku kepentingan.

Prasyarat

- Python dasar, SQL dasar, dan pemahaman data tabular.
- Mampu membaca diagram sederhana dan mengikuti contoh kode.
- Bersedia mencatat asumsi, kegagalan, dan keputusan selama latihan.

Cara Belajar yang Disarankan

1. Baca gambaran dan tujuan setiap bab.
2. Buat catatan istilah dengan bahasa sendiri.
3. Kerjakan checkpoint sebelum melihat kembali materi.
4. Jalankan atau adaptasi contoh kode bila relevan.
5. Gunakan mini project sebagai portofolio, bukan sekadar tugas akhir.
6. Lakukan review keamanan, privasi, dan dampak sebelum demonstrasi.

Peta Materi

Tahap	Fokus	Hasil
Fondasi	Istilah, tujuan, konteks, dan arsitektur	Peta masalah yang jelas
Pembangunan	Data, proses, komponen, dan integrasi	Baseline yang dapat diuji
Evaluasi	Kualitas, risiko, subgroup, biaya	Bukti dan daftar keterbatasan
Operasi	Monitoring, ownership, respons, iterasi	Sistem yang dapat dipelihara

Bab 1 - Peran dan Arsitektur Data Engineering

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran dan arsitektur data engineering dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Peran dan Arsitektur Data Engineering tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
sumber data	konsep penting dalam peran dan arsitektur data engineering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana sumber data diukur, diuji, atau dikendalikan?
ingestion	konsep penting dalam peran dan arsitektur data engineering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana ingestion diukur, diuji, atau dikendalikan?
storage	konsep penting dalam peran dan arsitektur data engineering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana storage diukur, diuji, atau dikendalikan?
transformation	konsep penting dalam peran dan arsitektur data engineering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana transformation diukur, diuji, atau dikendalikan?
serving	konsep penting dalam peran dan arsitektur data engineering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana serving diukur, diuji, atau dikendalikan?
konsumen data	konsep penting dalam peran dan arsitektur data engineering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana konsumen data diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **sumber data** -> **ingestion** -> **storage** -> **transformation**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **sumber data** dapat memengaruhi **ingestion**, lalu mengubah cara **storage** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan peran dan arsitektur data engineering dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk sumber data serta ingestion.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan peran dan arsitektur data engineering secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **sumber data** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan peran dan arsitektur data engineering.
- Menganggap sumber data sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan sumber data dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara ingestion dan storage dalam bab ini?
3. Sebutkan satu risiko ketika peran dan arsitektur data engineering diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan sumber data, ingestion, storage, transformation. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 2 - Sumber Data, Format, dan Data Contract

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran sumber data, format, dan data contract dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Sumber Data, Format, dan Data Contract tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
CSV	konsep penting dalam sumber data, format, dan data contract yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana CSV diukur, diuji, atau dikendalikan?
JSON	konsep penting dalam sumber data, format, dan data contract yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana JSON diukur, diuji, atau dikendalikan?
Parquet	konsep penting dalam sumber data, format, dan data contract yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana Parquet diukur, diuji, atau dikendalikan?
database	konsep penting dalam sumber data, format, dan data contract yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana database diukur, diuji, atau dikendalikan?
API	konsep penting dalam sumber data, format, dan data contract yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana API diukur, diuji, atau dikendalikan?
schema	aturan bentuk, tipe, dan makna field data	Pertanyaan yang perlu dijawab: bagaimana schema diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **CSV -> JSON -> Parquet -> database**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **CSV** dapat memengaruhi **JSON**, lalu mengubah cara **Parquet** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan sumber data, format, dan data contract dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk CSV serta JSON.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan sumber data, format, dan data contract secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **CSV** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan sumber data, format, dan data contract.
- Menganggap CSV sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan CSV dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara JSON dan Parquet dalam bab ini?
3. Sebutkan satu risiko ketika sumber data, format, dan data contract diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan CSV, JSON, Parquet, database. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 3 - Batch Processing dan Stream Processing

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran batch processing dan stream processing dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Batch Processing dan Stream Processing tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
batch	konsep penting dalam batch processing dan stream processing yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana batch diukur, diuji, atau dikendalikan?
event	konsep penting dalam batch processing dan stream processing yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana event diukur, diuji, atau dikendalikan?
stream	konsep penting dalam batch processing dan stream processing yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana stream diukur, diuji, atau dikendalikan?
latency	waktu yang dibutuhkan sistem untuk menghasilkan respons	Pertanyaan yang perlu dijawab: bagaimana latency diukur, diuji, atau dikendalikan?
throughput	jumlah pekerjaan yang dapat diselesaikan per satuan waktu	Pertanyaan yang perlu dijawab: bagaimana throughput diukur, diuji, atau dikendalikan?
watermark	konsep penting dalam batch processing dan stream processing yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana watermark diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **batch** -> **event** -> **stream** -> **latency**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **batch** dapat memengaruhi **event**, lalu mengubah cara **stream** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan batch processing dan stream processing dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk batch serta event.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan batch processing dan stream processing secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **batch** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan batch processing dan stream processing.
- Menganggap batch sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan batch dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara event dan stream dalam bab ini?
3. Sebutkan satu risiko ketika batch processing dan stream processing diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan batch, event, stream, latency. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 4 - ETL, ELT, dan Transformasi Data

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran etl, elt, dan transformasi data dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

ETL, ELT, dan Transformasi Data tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
extract	konsep penting dalam etl, elt, dan transformasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana extract diukur, diuji, atau dikendalikan?
load	konsep penting dalam etl, elt, dan transformasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana load diukur, diuji, atau dikendalikan?
transform	konsep penting dalam etl, elt, dan transformasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana transform diukur, diuji, atau dikendalikan?
staging	konsep penting dalam etl, elt, dan transformasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana staging diukur, diuji, atau dikendalikan?
incremental load	konsep penting dalam etl, elt, dan transformasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana incremental load diukur, diuji, atau dikendalikan?
idempotency	sifat proses yang aman diulang tanpa menggandakan efek	Pertanyaan yang perlu dijawab: bagaimana idempotency diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **extract** -> **load** -> **transform** -> **staging**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **extract** dapat memengaruhi **load**, lalu mengubah cara **transform** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan etl, elt, dan transformasi data dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk extract serta load.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan etl, elt, dan transformasi data secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **extract** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan etl, elt, dan transformasi data.
- Menganggap extract sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan extract dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara load dan transform dalam bab ini?
3. Sebutkan satu risiko ketika etl, elt, dan transformasi data diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan extract, load, transform, staging. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 5 - Data Warehouse, Data Lake, dan Lakehouse

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran data warehouse, data lake, dan lakehouse dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Data Warehouse, Data Lake, dan Lakehouse tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
warehouse	konsep penting dalam data warehouse, data lake, dan lakehouse yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana warehouse diukur, diuji, atau dikendalikan?
lake	konsep penting dalam data warehouse, data lake, dan lakehouse yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana lake diukur, diuji, atau dikendalikan?
lakehouse	konsep penting dalam data warehouse, data lake, dan lakehouse yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana lakehouse diukur, diuji, atau dikendalikan?
partition	konsep penting dalam data warehouse, data lake, dan lakehouse yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana partition diukur, diuji, atau dikendalikan?
table format	konsep penting dalam data warehouse, data lake, dan lakehouse yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana table format diukur, diuji, atau dikendalikan?
medallion	konsep penting dalam data warehouse, data lake, dan lakehouse yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana medallion diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **warehouse -> lake -> lakehouse -> partition**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **warehouse** dapat memengaruhi **lake**, lalu mengubah cara **lakehouse** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan data warehouse, data lake, dan lakehouse dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk warehouse serta lake.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan data warehouse, data lake, dan lakehouse secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **warehouse** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan data warehouse, data lake, dan lakehouse.
- Menganggap warehouse sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan warehouse dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara lake dan lakehouse dalam bab ini?
3. Sebutkan satu risiko ketika data warehouse, data lake, dan lakehouse diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan warehouse, lake, lakehouse, partition. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 6 - Orkestrasi dan Penjadwalan Pipeline

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran orkestrasi dan penjadwalan pipeline dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Orkestrasi dan Penjadwalan Pipeline tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
DAG	konsep penting dalam orkestrasi dan penjadwalan pipeline yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana DAG diukur, diuji, atau dikendalikan?
dependency	konsep penting dalam orkestrasi dan penjadwalan pipeline yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana dependency diukur, diuji, atau dikendalikan?
scheduler	konsep penting dalam orkestrasi dan penjadwalan pipeline yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana scheduler diukur, diuji, atau dikendalikan?
retry	konsep penting dalam orkestrasi dan penjadwalan pipeline yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana retry diukur, diuji, atau dikendalikan?
backfill	konsep penting dalam orkestrasi dan penjadwalan pipeline yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana backfill diukur, diuji, atau dikendalikan?
SLA	konsep penting dalam orkestrasi dan penjadwalan pipeline yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana SLA diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **DAG** -> **dependency** -> **scheduler** -> **retry**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **DAG** dapat memengaruhi **dependency**, lalu mengubah cara **scheduler** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan orkestrasi dan penjadwalan pipeline dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk DAG serta dependency.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan orkestrasi dan penjadwalan pipeline secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **DAG** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan orkestrasi dan penjadwalan pipeline.
- Menganggap DAG sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan DAG dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara dependency dan scheduler dalam bab ini?
3. Sebutkan satu risiko ketika orkestrasi dan penjadwalan pipeline diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan DAG, dependency, scheduler, retry. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 7 - Kualitas Data dan Pengujian

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran kualitas data dan pengujian dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Kualitas Data dan Pengujian tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
validity	konsep penting dalam kualitas data dan pengujian yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana validity diukur, diuji, atau dikendalikan?
completeness	konsep penting dalam kualitas data dan pengujian yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana completeness diukur, diuji, atau dikendalikan?
uniqueness	konsep penting dalam kualitas data dan pengujian yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana uniqueness diukur, diuji, atau dikendalikan?
freshness	konsep penting dalam kualitas data dan pengujian yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana freshness diukur, diuji, atau dikendalikan?
schema test	konsep penting dalam kualitas data dan pengujian yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana schema test diukur, diuji, atau dikendalikan?
reconciliation	konsep penting dalam kualitas data dan pengujian yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana reconciliation diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **validity** -> **completeness** -> **uniqueness** -> **freshness**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **validity** dapat memengaruhi **completeness**, lalu mengubah cara **uniqueness** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan kualitas data dan pengujian dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk validity serta completeness.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan kualitas data dan pengujian secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **validity** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan kualitas data dan pengujian.
- Menganggap validity sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan validity dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara completeness dan uniqueness dalam bab ini?
3. Sebutkan satu risiko ketika kualitas data dan pengujian diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan validity, completeness, uniqueness, freshness. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 8 - Metadata, Lineage, dan Governance

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran metadata, lineage, dan governance dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Metadata, Lineage, dan Governance tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
catalog	konsep penting dalam metadata, lineage, dan governance yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana catalog diukur, diuji, atau dikendalikan?
lineage	jejak asal dan transformasi data atau artefak	Pertanyaan yang perlu dijawab: bagaimana lineage diukur, diuji, atau dikendalikan?
ownership	konsep penting dalam metadata, lineage, dan governance yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana ownership diukur, diuji, atau dikendalikan?
classification	konsep penting dalam metadata, lineage, dan governance yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana classification diukur, diuji, atau dikendalikan?
retention	konsep penting dalam metadata, lineage, dan governance yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana retention diukur, diuji, atau dikendalikan?
audit	konsep penting dalam metadata, lineage, dan governance yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana audit diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **catalog** -> **lineage** -> **ownership** -> **classification**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **catalog** dapat memengaruhi **lineage**, lalu mengubah cara **ownership** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan metadata, lineage, dan governance dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk catalog serta lineage.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan metadata, lineage, dan governance secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **catalog** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan metadata, lineage, dan governance.
- Menganggap catalog sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan catalog dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara lineage dan ownership dalam bab ini?
3. Sebutkan satu risiko ketika metadata, lineage, dan governance diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan catalog, lineage, ownership, classification. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 9 - Keamanan dan Privasi Data

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran keamanan dan privasi data dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Keamanan dan Privasi Data tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
least privilege	pemberian akses minimum yang benar-benar diperlukan	Pertanyaan yang perlu dijawab: bagaimana least privilege diukur, diuji, atau dikendalikan?
encryption	konsep penting dalam keamanan dan privasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana encryption diukur, diuji, atau dikendalikan?
masking	konsep penting dalam keamanan dan privasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana masking diukur, diuji, atau dikendalikan?
secret	konsep penting dalam keamanan dan privasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana secret diukur, diuji, atau dikendalikan?
access control	konsep penting dalam keamanan dan privasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana access control diukur, diuji, atau dikendalikan?
privacy	konsep penting dalam keamanan dan privasi data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana privacy diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **least privilege** -> **encryption** -> **masking** -> **secret**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **least privilege** dapat memengaruhi **encryption**, lalu mengubah cara **masking** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan keamanan dan privasi data dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk least privilege serta encryption.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan keamanan dan privasi data secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **least privilege** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan keamanan dan privasi data.
- Menganggap least privilege sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan least privilege dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara encryption dan masking dalam bab ini?
3. Sebutkan satu risiko ketika keamanan dan privasi data diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan least privilege, encryption, masking, secret. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 10 - Observability, Reliability, dan Biaya

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran observability, reliability, dan biaya dalam Data Engineering;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Observability, Reliability, dan Biaya tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Data engineer seperti perancang sistem air kota: bukan hanya memindahkan air, tetapi memastikan sumber, pipa, penyimpanan, tekanan, kebersihan, dan pemantauan bekerja bersama.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
logging	konsep penting dalam observability, reliability, dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana logging diukur, diuji, atau dikendalikan?
metric	ukuran terdefinisi yang digunakan untuk memantau kondisi atau hasil	Pertanyaan yang perlu dijawab: bagaimana metric diukur, diuji, atau dikendalikan?
alert	konsep penting dalam observability, reliability, dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana alert diukur, diuji, atau dikendalikan?
incident	konsep penting dalam observability, reliability, dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana incident diukur, diuji, atau dikendalikan?
capacity	konsep penting dalam observability, reliability, dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana capacity diukur, diuji, atau dikendalikan?
FinOps	konsep penting dalam observability, reliability, dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana FinOps diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **logging** -> **metric** -> **alert** -> **incident**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **logging** dapat memengaruhi **metric**, lalu mengubah cara **alert** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan observability, reliability, dan biaya dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk logging serta metric.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Pipeline Analitik Penjualan**, tim perlu menerapkan observability, reliability, dan biaya secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **logging** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan observability, reliability, dan biaya.
- Menganggap logging sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan logging dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara metric dan alert dalam bab ini?
3. Sebutkan satu risiko ketika observability, reliability, dan biaya diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan logging, metric, alert, incident. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 11 - Mini Project: Pipeline Analitik Penjualan

Tujuan Proyek

Membangun pipeline batch yang mengambil data transaksi, memvalidasi skema, membersihkan duplikasi, membuat tabel analitik, dan menghasilkan laporan kualitas.

Proyek harus cukup kecil untuk selesai, tetapi cukup lengkap untuk memperlihatkan alur berpikir. Nilai utama bukan pada tampilan atau kompleksitas, melainkan kejelasan tujuan, kualitas keputusan, pengujian, dan dokumentasi.

Deliverable

- problem statement dan intended use;
- diagram arsitektur atau alur kerja;
- data dictionary atau inventaris sumber;
- baseline yang dapat dijalankan;
- hasil pengujian normal, error, dan kasus batas;
- daftar risiko serta kontrol;
- ringkasan hasil untuk audiens nonteknis;
- README dan rencana pengembangan.

Tahapan Pengerjaan

1. Tulis masalah, pengguna, keputusan, dan batas penggunaan.
2. Buat inventaris data, sumber daya, pemilik, izin, serta risiko.
3. Rancang arsitektur atau alur kerja versi minimum.
4. Bangun baseline yang dapat dijalankan ulang.
5. Tambahkan validasi, logging, dan penanganan error.
6. Susun golden set atau skenario uji, termasuk kasus batas.
7. Ukur outcome, guardrail, performa, dan biaya.
8. Dokumentasikan hasil, keterbatasan, runbook, serta rencana iterasi.

Contoh Kode Awal

```
from pathlib import Path
import pandas as pd

REQUIRED = {"transaction_id", "customer_id", "amount", "created_at"}

def validate_sales(path: str) -> pd.DataFrame:
    df = pd.read_csv(Path(path))
    missing = REQUIRED - set(df.columns)
    if missing:
        raise ValueError(f"Kolom wajib hilang: {sorted(missing)}")
    if df["transaction_id"].duplicated().any():
        raise ValueError("transaction_id harus unik")
    if (df["amount"] < 0).any():
        raise ValueError("amount tidak boleh negatif")
    df["created_at"] = pd.to_datetime(df["created_at"], errors="raise")
    return df
```

Kode tersebut hanya titik awal. Tambahkan pemeriksaan input, konfigurasi, test, logging, serta penanganan error sesuai konteks proyek.

Rubrik Penilaian

Aspek	Bobot	Indikator
Problem framing	15%	Pengguna, keputusan, outcome, dan batas jelas
Data atau input	15%	Sumber, izin, kualitas, dan representasi dijelaskan
Rancangan	15%	Komponen, interface, serta trade-off

Aspek	Bobot	Indikator
		masuk akal
Implementasi	20%	Baseline berjalan, modular, dan dapat diulang
Evaluasi	15%	Metrik, kasus batas, serta subgroup diperiksa
Responsible practice	10%	Privasi, keamanan, fairness, dan human review
Komunikasi	10%	Dokumentasi dan demo mudah dipahami

Pertanyaan Review Proyek

1. Keputusan apa yang benar-benar dibantu proyek ini?
2. Apa kegagalan yang paling mungkin dan paling berbahaya?
3. Bukti apa yang menunjukkan solusi lebih baik daripada baseline?
4. Siapa yang berwenang menyetujui, menghentikan, atau mengubah sistem?
5. Informasi apa yang harus terlihat oleh pengguna agar tidak salah memahami hasil?

Bab 12 - Kuis Akhir

Pilih jawaban yang paling tepat.

1. Ketika memulai **Peran dan Arsitektur Data Engineering**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji sumber data
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
2. Ketika memulai **Sumber Data, Format, dan Data Contract**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji CSV
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
3. Ketika memulai **Batch Processing dan Stream Processing**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji batch
 - D. Mengukur keberhasilan hanya dari jumlah fitur
4. Ketika memulai **ETL, ELT, dan Transformasi Data**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji extract
5. Ketika memulai **Data Warehouse, Data Lake, dan Lakehouse**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji warehouse
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
6. Ketika memulai **Orkestrasi dan Penjadwalan Pipeline**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji DAG
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
7. Ketika memulai **Kualitas Data dan Pengujian**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji validity
 - D. Mengukur keberhasilan hanya dari jumlah fitur
8. Ketika memulai **Metadata, Lineage, dan Governance**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji catalog
9. Ketika memulai **Keamanan dan Privasi Data**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji least privilege
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
10. Ketika memulai **Observability, Reliability, dan Biaya**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji logging
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur

11. Input penting berubah tanpa pemberitahuan. Kontrol terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Kontrak, validasi, versioning, dan alert
 - D. menghapus logging untuk menghemat ruang
12. Hasil sistem terlihat baik pada data latihan tetapi buruk pada konteks baru. Langkah terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Memeriksa split, leakage, representasi data, dan generalisasi
13. Tim tidak tahu siapa yang harus merespons alarm. Perbaikan utama adalah ...
- A. Menetapkan ownership, severity, runbook, dan escalation
 - B. menambah kompleksitas tanpa diagnosis
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
14. Perubahan besar akan diluncurkan. Cara menurunkan blast radius adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. Rilis bertahap, observasi, dan rollback
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
15. Pengguna tidak memahami keterbatasan hasil. Desain yang tepat adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Menjelaskan sumber, ketidakpastian, batas penggunaan, dan koreksi
 - D. menghapus logging untuk menghemat ruang
16. Data sensitif digunakan untuk latihan. Prinsip pertama adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Minimization, izin, kontrol akses, dan penggunaan data sintetis bila memungkinkan
17. Satu metrik meningkat tetapi dampak keseluruhan memburuk. Tim perlu ...
- A. Menggunakan outcome metric dan guardrail secara bersamaan
 - B. menambah kompleksitas tanpa diagnosis
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
18. Sistem gagal pada sebagian kelompok. Respons yang benar adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. Menganalisis subgroup, dampak, akar masalah, dan mitigasi
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
19. Biaya naik tanpa kenaikan nilai. Langkah awal adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Mengukur penggunaan per komponen dan menghubungkannya dengan outcome
 - D. menghapus logging untuk menghemat ruang
20. Temuan belum cukup kuat untuk keputusan otomatis. Pilihan aman adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Membatasi penggunaan sebagai dukungan keputusan dan menambahkan human review

Kunci dan Pembahasan

No.	Jawaban	Pembahasan
1	A	Pendekatan dimulai dari tujuan dan

No.	Jawaban	Pembahasan
		definisi operasional sumber data, kemudian alat dipilih sesuai kebutuhan.
2	B	Pendekatan dimulai dari tujuan dan definisi operasional CSV, kemudian alat dipilih sesuai kebutuhan.
3	C	Pendekatan dimulai dari tujuan dan definisi operasional batch, kemudian alat dipilih sesuai kebutuhan.
4	D	Pendekatan dimulai dari tujuan dan definisi operasional extract, kemudian alat dipilih sesuai kebutuhan.
5	A	Pendekatan dimulai dari tujuan dan definisi operasional warehouse, kemudian alat dipilih sesuai kebutuhan.
6	B	Pendekatan dimulai dari tujuan dan definisi operasional DAG, kemudian alat dipilih sesuai kebutuhan.
7	C	Pendekatan dimulai dari tujuan dan definisi operasional validity, kemudian alat dipilih sesuai kebutuhan.
8	D	Pendekatan dimulai dari tujuan dan definisi operasional catalog, kemudian alat dipilih sesuai kebutuhan.
9	A	Pendekatan dimulai dari tujuan dan definisi operasional least privilege, kemudian alat dipilih sesuai kebutuhan.
10	B	Pendekatan dimulai dari tujuan dan definisi operasional logging, kemudian alat dipilih sesuai kebutuhan.
11	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
12	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
13	A	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
14	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
15	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
16	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
17	A	Pilihan tersebut membuat masalah

No.	Jawaban	Pembahasan
		dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
18	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
19	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
20	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.

Interpretasi Skor

- **17-20 benar:** siap mengerjakan proyek dengan pengawasan ringan.
- **13-16 benar:** pemahaman baik, ulangi bagian yang masih lemah.
- **9-12 benar:** pelajari kembali konsep inti dan latihan.
- **0-8 benar:** ulangi modul secara bertahap dengan mentor atau teman belajar.

Bab 13 - Diskusi dan Refleksi

Pertanyaan Diskusi

1. Bagian mana dari Data Engineering yang paling mudah diremehkan tetapi berpotensi menimbulkan kegagalan besar?
2. Kapan solusi sederhana lebih baik daripada solusi yang lebih otomatis atau kompleks?
3. Metrik apa yang dapat mendorong perilaku salah bila digunakan sendirian?
4. Bagaimana pengguna dapat mengoreksi sistem dan melihat tindak lanjutnya?
5. Siapa yang menerima manfaat dan siapa yang mungkin menerima risiko?

Jurnal Refleksi

Tuliskan satu halaman yang menjawab:

- konsep yang paling mengubah cara berpikir;
- asumsi yang sebelumnya dianggap benar;
- kesalahan yang terjadi ketika latihan;
- bukti yang masih kurang;
- satu tindakan belajar berikutnya.

Bab 14 - Glosarium dan Checklist

Glosarium

Istilah	Makna ringkas
sumber data	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
ingestion	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
storage	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
transformation	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
serving	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
konsumen data	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
CSV	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
JSON	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
Parquet	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
database	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
API	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
schema	aturan bentuk, tipe, dan makna field data.
data contract	kesepakatan eksplisit antara produsen dan konsumen data.
batch	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
event	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
stream	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
latency	waktu yang dibutuhkan sistem untuk menghasilkan respons.
throughput	jumlah pekerjaan yang dapat diselesaikan per satuan waktu.
watermark	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
extract	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
load	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
transform	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
staging	konsep penting dalam data engineering yang perlu diberi

Istilah	Makna ringkas
	definisi operasional sebelum dipakai.
incremental load	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
idempotency	sifat proses yang aman diulang tanpa menggandakan efek.
warehouse	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
lake	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
lakehouse	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
partition	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
table format	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
medallion	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
DAG	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
dependency	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
scheduler	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
retry	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
backfill	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
SLA	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
validity	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
completeness	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
uniqueness	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
freshness	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
schema test	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
reconciliation	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
catalog	konsep penting dalam data engineering yang perlu diberi definisi operasional sebelum dipakai.
lineage	jejak asal dan transformasi data atau artefak.

Checklist Sebelum Implementasi

- [] Masalah, pengguna, dan keputusan telah dinyatakan.
- [] Intended use dan penggunaan yang dilarang telah ditulis.

- [] Sumber data atau input memiliki izin dan provenance.
- [] Baseline dan kriteria penerimaan tersedia.
- [] Kondisi error, data kosong, dan kasus ekstrem diuji.
- [] Privasi, keamanan, bias, dan aksesibilitas ditinjau.
- [] Log, metric, owner, alert, dan runbook ditetapkan.
- [] Perubahan memiliki versioning, approval, dan rollback.
- [] Hasil dikomunikasikan bersama keterbatasannya.
- [] Jadwal review dan penghentian sistem ditentukan.

Bab 15 - Ringkasan dan Referensi

Ringkasan Cepat

1. Mulai dari keputusan dan pengguna, bukan alat.
2. Definisikan istilah, data, outcome, dan guardrail secara operasional.
3. Bangun baseline kecil yang dapat diamati dan diuji ulang.
4. Perlakukan keamanan, privasi, aksesibilitas, serta human review sebagai bagian desain.
5. Uji kondisi normal, kegagalan, subgroup, perubahan konteks, dan biaya.
6. Dokumentasikan provenance, asumsi, versi, owner, serta keterbatasan.
7. Gunakan monitoring dan feedback untuk memutuskan iterasi, rollback, atau penghentian.

Referensi Utama

- The Data Warehouse Toolkit - Ralph Kimball dan Margy Ross.
- Designing Data-Intensive Applications - Martin Kleppmann.
- Dokumentasi resmi Apache Airflow.
- Dokumentasi resmi Apache Kafka.
- Dokumentasi resmi dbt tentang analytics engineering.

Penutup

Kemampuan dalam Data Engineering tidak hanya ditunjukkan oleh banyaknya alat yang dikuasai. Kemampuan terlihat dari cara peserta merumuskan masalah, memilih pendekatan yang proporsional, menguji klaim, melindungi pengguna, dan menjaga sistem tetap dapat dipertanggungjawabkan.

HerAI Fellowship - Participant Module Versi 1.0.0 - 23 Juli 2026